Java Day 12：抽象类与接口

性能测试工程师学习计划 | Java基础进阶

一、今日学习目标

1. 理解抽象类、抽象方法以及 abstract 的使用方式。
2. 理解接口、interface 和 implements。
3. 区分抽象类与接口，知道各自适合解决的问题。
4. 理解单继承与多接口实现。
5. 能使用抽象类和接口设计性能测试结果处理程序。

> 建议时间：抽象类15分钟；接口15分钟；差异与选择10分钟；实战15分钟；练习与复盘按课后时间完成。

二、抽象类与抽象方法

当父类知道子类必须具备某个行为，但无法确定具体实现时，可以把该行为声明为抽象方法，交给子类完成。

1. 抽象方法

```
public abstract void showResult();
```

抽象方法只有声明，没有方法体；使用 abstract 修饰，并以分号结束。抽象方法必须位于抽象类或接口中。

2. 抽象类

```
abstract class ApiResult {

    protected int responseTime;
    protected int statusCode;

    public ApiResult(int responseTime, int statusCode) {
        this.responseTime = responseTime;
        this.statusCode = statusCode;
    }

    public boolean isSuccess() {
        return statusCode >= 200 && statusCode < 300;
    }

    public abstract void showResult();
}
```

抽象类可以包含普通属性、构造方法、普通方法和抽象方法。抽象类不能直接实例化；子类创建对象时会调用父类构造方法。

> 子类继承抽象类后，必须实现全部抽象方法；如果没有实现，子类也必须声明为 abstract。

3. 子类实现抽象方法

```
class LoginApiResult extends ApiResult {

    public LoginApiResult(int responseTime, int statusCode) {
        super(responseTime, statusCode);
    }

    @Override
    public void showResult() {
        System.out.println("Login API Result");
        System.out.println("Response Time: " + responseTime);
        System.out.println("Status Code: " + statusCode);
        System.out.println("Success: " + isSuccess());
    }
}
```

父类规定必须有 showResult()，子类决定具体如何显示。查询接口、下单接口也可以分别重写该方法。

三、接口 interface

接口可以理解为行为规范。例如 HTML、PDF、Excel 报告的生成方式不同，但都应该具备生成报告的能力。

1. 定义接口与实现接口

```
interface Reportable {
    void generateReport();
}

class HtmlReport implements Reportable {
    @Override
    public void generateReport() {
        System.out.println("Generate HTML Report");
    }
}

class PdfReport implements Reportable {
    @Override
    public void generateReport() {
        System.out.println("Generate PDF Report");
    }
}
```

类使用 implements 实现接口。实现类需要实现接口中的抽象方法，否则该实现类必须声明为 abstract。

2. 接口与多态

```
public class Main {
    public static void main(String[] args) {
        Reportable[] reports = {
            new HtmlReport(),
            new PdfReport()
        };

        for (Reportable report : reports) {
            report.generateReport();
        }
    }
}
```

Reportable 是接口类型，数组可以保存不同实现类的对象。调用 generateReport() 时，会执行实际对象对应的实现。这是 Day 11 多态的延伸。

3. 一个类可以实现多个接口

```
interface Reportable {
    void generateReport();
}

interface Exportable {
    void exportData();
}

class HtmlReport implements Reportable, Exportable {
    @Override
    public void generateReport() {
        System.out.println("Generate HTML Report");
    }

    @Override
    public void exportData() {
        System.out.println("Export HTML Data");
    }
}
```

Java 一个类只能直接继承一个父类，但可以实现多个接口。

四、抽象类与接口的区别

对比项	抽象类	接口
定义	abstract class	interface
使用方式	extends	implements
能否直接实例化	不能	不能
构造方法	可以有	没有普通构造方法
实例状态	可以有普通实例属性	字段默认 public static final
方法	普通方法、抽象方法	抽象方法，也支持 default、static 方法
数量	一个类只能继承一个类	一个类可以实现多个接口

> 选择建议：有共同属性、状态和部分公共实现时，考虑抽象类；只要求不同类遵循相同行为规范时，考虑接口。
> 一个类可以继承抽象父类，同时实现多个接口。

五、性能测试场景实战：统一结果与报告能力

设计目标：不同 API 结果共享响应时间、状态码和成功判断；每种 API 自己实现结果展示；报告对象统一具备生成报告能力。

```
abstract class ApiResult {
    protected int responseTime;
    protected int statusCode;

    public ApiResult(int responseTime, int statusCode) {
        this.responseTime = responseTime;
        this.statusCode = statusCode;
    }

    public boolean isSuccess() {
        return statusCode >= 200 && statusCode < 300;
    }
```

```
    public abstract void showResult();
}

interface Reportable {
    void generateReport();
}

class LoginApiResult extends ApiResult implements Reportable {
    public LoginApiResult(int responseTime, int statusCode) {
        super(responseTime, statusCode);
    }

    @Override
    public void showResult() {
        System.out.println("Login API");
        System.out.println("RT: " + responseTime);
        System.out.println("Success: " + isSuccess());
    }

    @Override
    public void generateReport() {
        System.out.println("Generate login report");
    }
}

public class Main {
    public static void main(String[] args) {
        LoginApiResult result = new LoginApiResult(180, 200);
        result.showResult();
        result.generateReport();
    }
}
```

理解重点：ApiResult 负责共享数据和成功判断；showResult() 强制子类提供自己的展示方式；Reportable 表示对象还具备生成报告的能力。

> 注意：这是用于学习面向对象设计的简化模型。真实性能报告还需要处理样本集合、TPS、错误率、P95/P99、时间窗口和数据导出等内容。

六、课后练习

练习一：抽象类

创建抽象类 PerformanceResult：

- 属性：responseTime（响应时间）、statusCode（状态码）。
- 构造方法初始化两个属性。
- 普通方法 isSuccess()：状态码在 200～299（含200、不含300）时返回 true。
- 抽象方法 showResult()。

练习二：实现两个子类

创建 LoginResult 和 QueryResult，继承 PerformanceResult。两个子类都实现 showResult()，分别输出登录请求和查询请求的类型、响应时间、状态码及成功状态。

练习三：接口

创建接口 Exportable，声明 exportData()。让 LoginResult 和 QueryResult 同时实现 Exportable，并分别输出不同的导出提示。

练习四：多态

在 Main 中创建 PerformanceResult 数组，放入一个 LoginResult 和一个 QueryResult，使用 for-each 循环调用 showResult()。再分别调用两个对象的 exportData()，体会接口引用的作用。

七、思考题与验收标准

思考题：

1. 抽象类为什么不能直接创建对象？
2. 抽象类能不能有构造方法？为什么？
3. 子类继承抽象类后，如果没有实现抽象方法会怎样？
4. 一个类为什么可以实现多个接口，却不能直接继承多个类？
5. 抽象类和接口分别适合解决什么问题？
6. PerformanceResult 数组为什么能够保存 LoginResult 和 QueryResult？

> 验收标准：
> - 能准确解释抽象类、抽象方法和接口。
> - 能正确使用 extends、implements、@Override。
> - 两个子类能正确实现抽象方法和接口方法。
> - 能用父类数组实现多态调用。
> - 代码能够编译运行，输出与设计一致。
> - 能回答至少 5 道思考题。

提交方式：将 PerformanceResult.java、LoginResult.java、QueryResult.java、Exportable.java、Main.java 上传，并附上控制台输出和思考题答案。验收通过后再进入 Day 13。

> 今日关键记忆：抽象类强调“共同基础与部分实现”；接口强调“行为能力与规范”；多态让统一类型调用不同实现。